

UT01 – Tecnologías de programación sobre clientes web

ACTIVIDAD EVALUABLE (RA1) – PLANTILLA DE ENTREGA

Nombre y Apellidos del alumno/a:

Rubén de la Torre Serrano

Fecha de entrega:

18/09/2026

URL del repositorio GitHub:

<https://github.com/delatorresmr1m/ACTIVIDADES-ENTREGABLES.git>

1. Repositorio y Estructura de la Actividad (0,50 pts)

El repositorio de GitHub ha sido configurado bajo la estructura obligatoria **ACTIVIDADES ENTREGABLES / ACT-1 /** y compartido con la profesora (aralvarezr@educa.jcyl.es). A continuación, se presentan las evidencias de código y funcionamiento de cada ejercicio.

 **Evidencia: Captura del explorador de archivos donde se vean todos los archivos generados.**

[Inserta aquí la captura de pantalla legible]

Ejercicio 1.1 – Carga Diferida con 'defer'

- Estructura: Archivo **index.html** con `<script defer src="..."></script>` ubicado en la cabecera `<head>`. El script genera los números del 1 al 100 en el cuerpo del documento mediante `document.body.innerHTML += "<p>"+i+"</p>"`.

Explica aquí qué ocurre si eliminas el atributo defer de la etiqueta `<script>`

Si se elimina el atributo 'defer', el script se carga antes que el cuerpo del HTML por lo que no se mostrará nada en el navegador.

Ejercicio 1.2 – Carga Asíncrona con 'async'

- Estructura: Archivo **index.html** con dos scripts asíncronos enlazados en el `<head>` mediante `<script async src="..."></script>`. Uno muestra los números del 1 al 100 y el otro del 900 al 1000.

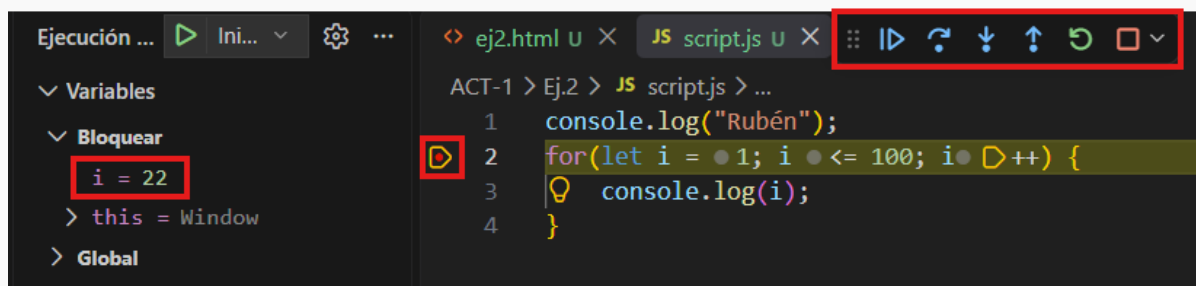
📷 Explica aquí qué se pretende demostrar y si se ha conseguido con este ejemplo

Lo que se pretende demostrar es, que mediante el atributo “async”, se puedan cargar los dos scripts en el HTML sin entrelazarse o solaparse y ejecutándose correctamente. Se ha conseguido con este ejemplo al cargar primero una serie de números y luego la otra.

2. Depuración en Visual Studio Code (0,75 pts)

La sesión de depuración se realiza sobre el archivo JavaScript ubicado en la carpeta **Ej.2**. A continuación se documentan los elementos clave de la interfaz de depuración de VS Code.

2.1. Punto de Ruptura (Breakpoint)

📷 Captura de Pantalla: Punto de ruptura (Breakpoint) activo en el código del Ej.2

Establecemos un punto de ruptura o Breakpoint en la línea de código deseada y procedemos a ejecutar la depuración mediante la interfaz de VS-Code, usando en panel flotante (situado en la parte superior de la primera captura) podremos ir saltando por cada breakpoint o por cada línea de Código, en este caso el depurador irá saltando al mismo breakpoint por cada iteración ya que está situado en un bucle “for”, en la zona izquierda de la captura podremos observar cómo va aumentando el valor de la variable “i” imprimiéndose por consola.

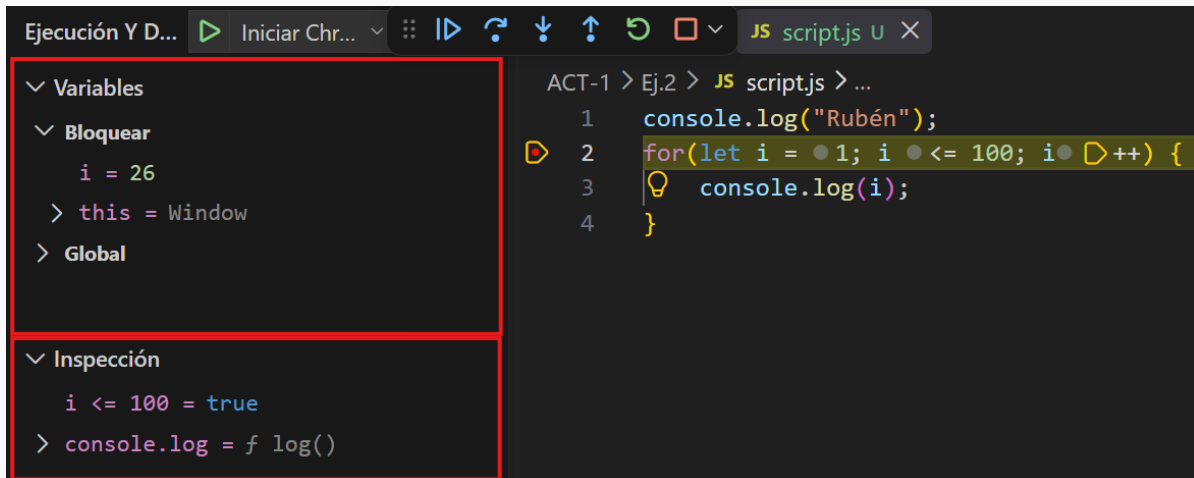
2.2. Botones de Control de Depuración de VS Code

Identificación y explicación técnica del efecto de los 6 botones del panel flotante de depuración:

Botón / Acción	Icono	Atajo de Teclado	Efecto / Descripción Técnica
Continuar (Continue)		F5	Sigue hasta el siguiente punto de interrupción.
Paso a paso por encima (Step Over)		F10	Ejecuta la línea sin entrar en sus funciones.
Paso a paso por dentro (Step Into)		F11	Entra en una función para ver qué hace por dentro.
Paso a paso para salir (Step Out)		Máyus+F11	Termina la función actual y vuelve al código que la llamó.
Reiniciar (Restart)		Ctrl+Máyus+F5	Vuelve a empezar la depuración.
Detener (Stop)		Máyus+F5	Termina la depuración.

2.3. Inspección de Código: Zonas Watch y Variables

 Captura de Pantalla: Panel 'Watch' con expresiones en seguimiento y 'Variables' en un estado concreto

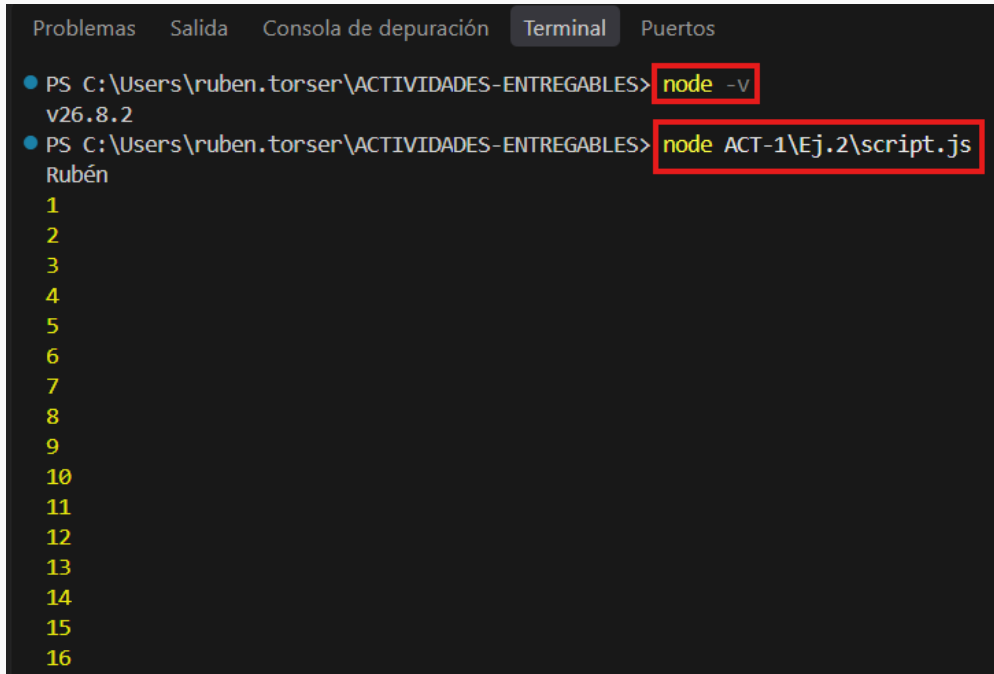


El panel Watch (Inspección) muestra el valor de las variables o expresiones que añades para vigilarlas mientras que el panel Variables muestra las variables disponibles y sus valores justo donde has pausado el programa.

3. Ejecución con Node.js (0,25 pts)

Ejecución del script de la carpeta Ej2 en el entorno de servidor local Node.js mediante la terminal integrada de Visual Studio Code.

 **Captura de Pantalla: Terminal integrada con comando 'node -v', la ruta completa y la ejecución del script mostrando la salida personalizada**



```
Problemas  Salida  Consola de depuración  Terminal  Puertos
• PS C:\Users\ruben.torser\ACTIVIDADES-ENTREGABLES> node -v
v26.8.2
• PS C:\Users\ruben.torser\ACTIVIDADES-ENTREGABLES> node ACT-1\Ej.2\script.js
Rubén
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
```

Node.js nos permite ejecutar JavaScript fuera del navegador, para ejecutar un script, escribimos “node” en la consola y a continuación la ruta del archivo js para que se muestree el resultado.

4. Lista de Cotejo / Autoevaluación

Calificación máxima: 1,50 pts · Escala: Sí = puntuación total · Parcial = 50 % · No = 0

Criterio de Evaluación	Puntuación Máx.	Sí/Parcial /No
1. Repositorio y estructura	0,50	Sí
· Repositorio compartido con aralvarezr@educa.jcyl.es y accesible (sin 404).	0,05	Sí
· Carpeta ACTIVIDADES ENTREGABLES / ACT-1 con el documento y ejercicios.	0,05	Sí
· Ej.0.0: .html con <script src="..."> y script.js que muestra 'Hola mundo'.	0,10	Sí
· Ej.0.1: .html con script en línea (sin src) y resultado coincidente con Ej.0.0.	0,10	Sí
· Ej.1.1: Script externo con defer en <head> escribe 1-100 en document.body.	0,10	Sí
· Ej.1.2: Dos scripts con async en <head>, uno 1-100 y otro 900-1000.	0,10	Sí
2. Depuración en Visual Studio Code	0,75	Sí
· Puntos de ruptura (breakpoints) con ejecución pausada y línea resaltada.	0,20	Sí
· 6 Botones de avance identificados y explicados con sus efectos.	0,20	Sí
· Zona Watch con expresión añadida y valor durante la ejecución.	0,15	Sí
· Zona Variables comentando cambios de valor en ejecución.	0,15	Sí
· Calidad de evidencia: capturas legibles.	0,05	Sí
3. Ejecución con Node.js	0,25	Sí
· Ejecución del commando en la ruta correcta.	0,10	Sí
· Salida por consola correcta, incluyendo el nombre del alumno	0,10	Sí
· Entorno visible: terminal integrada de VS Code y versión de Node (node -v).	0,05	Sí
CALIFICACIÓN OBTENIDA		1,50

Observaciones:

[Si quieres añadir alguna observación sobre esta actividad, puedes hacerlo aquí]